

Projet Machine Learning

Analyse Comportementale Clientèle Retail

E-commerce de Cadeaux – Chaîne Complète de Traitement

Filière GI2 – Module Machine Learning

Préparé par

Amine Hizi

Document pédagogique

Année Universitaire 2025–2026

Résumé

Ce rapport présente la chaîne complète de traitement d'un projet de Machine Learning appliqué à l'analyse comportementale de la clientèle d'un e-commerce de cadeaux. L'objectif principal est de comprendre et segmenter le comportement des clients à travers deux techniques fondamentales : le clustering non supervisé pour la segmentation RFM (Recency, Frequency, Monetary), et la classification supervisée pour la prédiction du churn (taux de désabonnement).

Le projet couvre l'ensemble du pipeline data science moderne, depuis l'exploration des données brutes imparfaites jusqu'au déploiement opérationnel via une interface web Flask accessible aux équipes métier. Ce travail met particulièrement l'accent sur la qualité des données, la prévention du data leakage, et l'interprétabilité des modèles pour une adoption en entreprise.

Mots-clés : Machine Learning, Churn Prediction, RFM Segmentation, Random Forest, K-Means, Flask API, Data Preprocessing, Retail Analytics

Table des matières

1	Introduction Générale	3
1.1	Contexte et Motivation	3
1.2	Objectifs du Projet	3
1.3	Présentation du Jeu de Données	3
1.4	Structure des 52 Features	3
1.4.1	Features Numériques (1–34)	4
1.4.2	Features Catégorielles (35–52)	4
2	Architecture et Organisation du Projet	6
2.1	Structure du Repository Git	6
2.2	Environnement Virtuel et Dépendances	6
3	Exploration des Données (EDA)	8
3.1	Objectifs de l'Exploration	8
3.2	Problèmes de Qualité Identifiés	8
3.3	Anomalies Spécifiques Détectées	8
3.4	Analyse de la Distribution du Churn	9
4	Préparation des Données (Preprocessing)	10
4.1	Enjeu Principal : Le Data Leakage	10
4.2	Identification des Features à Risque	10
4.3	Pipeline de Preprocessing Complet	10
4.4	Imputation des Valeurs Manquantes	11
4.5	Encodage des Variables Catégorielles	11
5	Modélisation : Les Deux Techniques	12
5.1	Technique 1 : Clustering Non Supervisé (K-Means)	12
5.1.1	Objectif et Principe	12
5.1.2	Algorithme K-Means	12
5.1.3	Résultats et Interprétation	12
5.2	Technique 2 : Classification Supervisée (Random Forest)	13
5.2.1	Objectif et Principe	13
5.2.2	Choix de l'Algorithme : Random Forest	13
5.2.3	Évaluation des Performances	14
5.2.4	Importance des Features	14
6	Déploiement : Interface Flask	15
6.1	Architecture de l'Application Web	15
6.2	Chargement des Modèles en Production	15
6.3	Mode Simplifié : 6 Features Essentielles	15
6.4	Exemple de Résultat d'Analyse	16
7	Compétences Acquises et Mapping Pédagogique	17
8	Conclusion et Perspectives	18
8.1	Bilan du Projet	18
8.2	Limites et Améliorations Futures	18
8.3	Apports Pédagogiques	18

Introduction Générale

Contexte et Motivation

Dans un environnement e-commerce hyper-concurrentiel, la rétention client représente un enjeu stratégique majeur. De nombreuses études marketing démontrent que l'acquisition d'un nouveau client coûte 5 à 25 fois plus cher que la fidélisation d'un client existant. Pour une entreprise de cadeaux en ligne, comprendre qui part, quand et pourquoi, permet d'intervenir proactivement avec des offres personnalisées et d'optimiser le chiffre d'affaires.

Ce projet s'inscrit dans une démarche de Customer Relationship Management (CRM) analytique, où les techniques de Machine Learning viennent renforcer l'intelligence métier traditionnelle. L'approche adoptée combine des méthodes non supervisées pour la découverte de segments naturels dans les données, et des méthodes supervisées pour la prédiction de comportements futurs.

Objectifs du Projet

L'entreprise e-commerce de cadeaux nous confie une mission data science structurée autour de deux objectifs stratégiques principaux :

1. **Personnaliser les stratégies marketing** : Adapter les campagnes selon le profil comportemental de chaque client, en identifiant des segments homogènes via la segmentation RFM.
2. **Réduire le taux de départ des clients (churn)** : Détecter précocement les signaux de désengagement pour intervenir avant le départ définitif, grâce à un modèle de classification supervisée.

Présentation du Jeu de Données

Le dataset fourni, nommé `retail_customers_COMPLETE_CATEGORICAL`, est un fichier Excel (.xlsx) contenant des données transactionnelles et des informations complémentaires sur la clientèle. Il s'agit d'un dataset intentionnellement imparfait, conçu pédagogiquement pour permettre de maîtriser l'ensemble des étapes de nettoyage et de préparation.

TABLE 1 – Caractéristiques générales du dataset

Caractéristique	Valeur
Nombre de clients (lignes)	4 372
Nombre de features (colonnes)	52
Format du fichier	Excel (.xlsx)
Variable cible	Churn (binaire : 0 ou 1)

La répartition de la variable cible révèle un déséquilibre classique en marketing :

- Clients fidèles (Churn = 0) : 2 918 clients, soit **66,7%**
- Clients partis (Churn = 1) : 1 454 clients, soit **33,3%**

Ce ratio d'environ 2 :1 n'est pas extrêmement déséquilibré, mais nécessite néanmoins une attention particulière lors de l'entraînement des modèles.

Structure des 52 Features

Les features sont organisées en deux grands groupes selon leur nature.

Features Numériques (1–34)

Ces variables quantitatives décrivent les comportements d’achat, les patterns temporels et les caractéristiques démographiques. Parmi les plus importantes :

TABLE 2: Sélection de features numériques clés

N°	Feature	Intervalle	Description
1	CustomerID	10 000–99 999	Identifiant unique client
2	Recency	0–400	Jours écoulés depuis dernier achat
3	Frequency	1–50	Nombre de commandes distinctes
4	MonetaryTotal	-5 000–15 000	Somme totale dépensée (£)
5	MonetaryAvg	5–500	Montant moyen par commande (£)
6	MonetaryStd	0–500	Écart-type des dépenses (£)
9	TotalQuantity	-10 000–100 000	Quantité totale d’articles
13	CustomerTenure	0–730	Durée relation client (jours)
18	WeekendRatio	0,0–1,0	Proportion achats weekend
19	AvgDaysBetween	0–365	Délai moyen entre commandes
26	CancelledTrans	0–50	Transactions annulées
27	ReturnRatio	0,0–1,0	Taux retour (quantités négatives)
31	Age	18–81 ou NaN	Âge estimé client (ans)
32	SupportTickets	-1, 0–15, 999	Tickets support ouverts
33	Satisfaction	-1, 0, 1–5, 99	Note satisfaction (/5)
34	Churn	0 ou 1	Indicateur départ client

Features Catégorielles (35–52)

Ces variables qualitatives décrivent les segments, préférences et caractéristiques socio-démographiques :

TABLE 3: Features catégorielles du dataset

N°	Feature	Cardinalité	Description / Valeurs
35	RFMSegment	4	Champions, Fidèles, Potentiels, Dormants
36	AgeCategory	7	18-24, 25-34, . . . , 65+, Inconnu
37	SpendingCat	4	Low, Medium, High, VIP
38	CustomerType	5	Hyperactif, Régulier, Occasionnel, Nouveau, Perdu
39	FavoriteSeason	4	Hiver, Printemps, Été, Automne
40	PreferredTime	5	Matin, Midi, Après-midi, Soir, Nuit
41	Region	8	UK, Europe (N/S/E/C), Asie, Autre
42	LoyaltyLevel	5	Nouveau, Jeune, Établi, Ancien, Inconnu
43	ChurnRisk	4	Faible, Moyen, Élevé, Critique
47	Gender	3	M, F, Unknown
48	AccountStatus	4	Active, Suspended, Pending, Closed

N°	Feature	Cardinalité	Description / Valeurs
49	Country	37+	UK, France, Germany, etc.
50	Newsletter	1	Yes (valeur unique)

Architecture et Organisation du Projet

Structure du Repository Git

Le projet suit l'architecture standard d'un projet Machine Learning en production, structurée en plusieurs répertoires distincts pour séparer les responsabilités :

Listing 1 – Arborescence complète du projet

```

1 projet_ml_retail/
2 |-- data/
3 |   |-- raw/
4 |   |   \-- retail_customers_COMPLETE_CATEGORICAL.xlsx
5 |   |-- processed/           # Donnees nettoyees et transformees
6 |   \-- train_test/
7 |       |-- X_train.csv
8 |       |-- X_test.csv
9 |       |-- y_train.csv
10 |       \-- y_test.csv
11 |
12 |-- notebooks/
13 |   \-- exploration.py       # Analyse exploratoire (EDA)
14 |
15 |-- src/                     # Scripts Python de production
16 |
17 |-- models/                  # Modeles serialises (artefacts)
18 |   |-- churn_classifier.pkl
19 |   |-- kmeans_model.pkl
20 |   |-- ltv_regressor.pkl
21 |   \-- preprocessor.pkl
22 |
23 |-- presentation/
24 |   \-- presentation.pptx
25 |
26 |-- rapport/
27 |
28 |-- reports/                  # Visualisations et rapports
29 |   |-- confusion_matrix.png
30 |   |-- feat_imp.png
31 |   \-- training_report.json
32 |
33 |-- requirements.txt
34 |-- README.md
35 \-- .gitignore

```

Environnement Virtuel et Dépendances

L'utilisation d'un environnement virtuel (`venv`) est obligatoire pour garantir la reproductibilité du projet. Le fichier `requirements.txt` liste toutes les dépendances avec leurs versions exactes :

Listing 2 – Fichier requirements.txt

```

1 # Data Science et ML
2 pandas==2.1.4
3 numpy==1.26.2
4 scikit-learn==1.3.2
5
6 # Visualisation

```

```
7 matplotlib==3.8.2
8 seaborn==0.13.0
9 plotly==5.17.0
10
11 # Boosting et optimisation
12 xgboost==2.0.2
13 lightgbm==4.1.0
14 optuna==3.4.0
15
16 # Deploiement web
17 flask==3.0.0
18 flask-cors==4.0.0
19
20 # Utilitaires
21 jupyter==1.0.0
22 joblib==1.3.2
23 openpyxl==3.1.2
24 python-dotenv==1.0.0
```

Listing 3 – Installation et génération des dépendances

```
1 pip install -r requirements.txt # Installation
2 pip freeze > requirements.txt # Generation
```


Exploration des Données (EDA)

Objectifs de l'Exploration

L'Analyse Exploratoire des Données (EDA – Exploratory Data Analysis) constitue la première étape cruciale du pipeline. Elle vise à :

1. **Comprendre la structure** : types de variables, dimensions, cardinalités
2. **Évaluer la qualité** : valeurs manquantes, aberrantes, inconsistances
3. **Détecter les patterns** : corrélations, distributions, relations cible-features
4. **Guider le preprocessing** : décisions informées sur les transformations nécessaires

Problèmes de Qualité Identifiés

L'exploration systématique a révélé plusieurs problèmes de qualité, conformément à l'intention pédagogique du dataset :

TABLE 4 – Synthèse des problèmes de qualité et traitements appliqués

Type de problème	Features concernées	Fréquence	Traitement appliqué
Valeurs manquantes	Age	30%	Imputation par la médiane (distribution asymétrique)
Valeurs aberrantes	SupportTickets, Satisfaction	5–8%	Détection IQR, capping aux bornes
Formats inconsistants	RegistrationDate	100%	Parsing avec <code>pd.to_datetime</code>
Feature inutile	NewsletterSubscribed	100% constante	Suppression
Données brutes	LastLoginIP	100%	Feature engineering, détection IP privée
Déséquilibre classes	Churn, AccountStatus	Variable	<code>class_weight='balanced'</code>

Anomalies Spécifiques Détectées

Au cours de l'exploration, une anomalie majeure a été découverte : certaines colonnes supposément numériques (`MonetaryTotal`, `MonetaryAvg`, `MonetaryStd`, `MonetaryMin`, `MonetaryMax`, `AvgDaysBetweenPurchases`) contenaient des dates au format chaîne de caractères au lieu de valeurs numériques.

Exemple d'anomalie :

- `MonetaryMin` : "2026-04-05 00:00:00" au lieu de 0.42
- `MonetaryAvg` : "2026-08-18 00:00:00" au lieu de 15.0

Cette corruption, probablement due à une erreur d'export Excel, a nécessité un nettoyage préalable :

Listing 4 – Nettoyage des types de données corrompus

```

1 # Detection des dates deguisees dans des champs numeriques
2 for col in numeric_columns:

```

```
3     if df[col].dtype == 'object':
4         sample_values = df[col].astype(str)
5
6         # Detecter les dates (contiennent un tiret et sont longues)
7         mask_date = sample_values.str.contains('-', na=False) & \
8             (sample_values.str.len() > 8)
9
10        # Remplacer les dates par 0 (valeur neutre)
11        df.loc[mask_date, col] = '0'
12
13        # Conversion finale en numerique
14        df[col] = pd.to_numeric(df[col], errors='coerce')
```

Analyse de la Distribution du Churn

La variable cible Churn présente une distribution déséquilibrée mais gérable. Avec 2 918 clients fidèles (66,7%) contre 1 454 clients partis (33,3%), ce ratio de 2 :1 ne nécessite pas de techniques avancées de rééquilibrage telles que SMOTE, mais impose l'utilisation du paramètre `class_weight='balanced'` lors de l'entraînement.

Préparation des Données (Preprocessing)

Enjeu Principal : Le Data Leakage

Le **data leakage** (fuite de données) est l'erreur la plus grave et la plus fréquente en Machine Learning. Il survient lorsque des features utilisées pour prédire la cible contiennent indirectement l'information qu'elles sont censées prédire.

Exemple concret de data leakage

Si nous conservons **ChurnRiskCategory** (qui vaut "Critique" quand Churn=1), le modèle n'apprend rien – il "triche" en lisant directement la réponse. Ce type d'erreur produit des performances artificiellement parfaites en évaluation mais un modèle totalement inutilisable en production.

Identification des Features à Risque

Une analyse méticuleuse a permis d'identifier **13 features** causant du leakage :

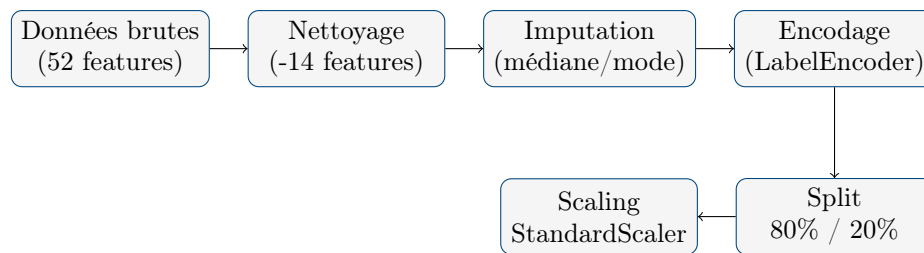
TABLE 5 – Features supprimées pour prévention du data leakage

Feature supprimée	Justification du leakage
ChurnRiskCategory	Dérivée directement de la variable cible (corrélation parfaite)
RFMSegment	Segment calculé a posteriori incluant l'information de churn
CustomerType	Contient la catégorie "Perdu" indiquant explicitement un client parti
AccountStatus	Valeur "Closed" = client déjà parti
SatisfactionScore	Scores négatifs (-1) ou très élevés (99) sont des codes d'erreur liés au départ
SupportTicketsCount	Valeur 999 est un code interne indiquant un client résilié
AgeCategory	Dérivée de Age (redondance)
LoyaltyLevel	Dérivée de CustomerTenure (redondance)
WeekendPreference	Dérivée de WeekendPurchaseRatio (redondance)
CustomerID	Identifiant unique sans valeur prédictive
LastLoginIP	Identifiant unique, information non généralisable
RegistrationDate	Formats inconsistants, traitée séparément si besoin
NewsletterSubscribed	Feature constante (100% "Yes"), variance nulle

Après suppression, il reste **38 features propres** utilisables pour la modélisation.

Pipeline de Préprocessing Complet

Le pipeline suit le processus industriel standard, avec une attention particulière à la séparation train/test afin d'éviter toute contamination de l'ensemble de test :



Point critique : ordre du scaling

Le `StandardScaler` est fit uniquement sur `X_train`, puis `transform` appliqué sur `X_test`. Faire `fit_transform` sur tout le dataset avant le split constituerait une fuite de données.

Listing 5 – Scaling correct sans data leakage

```

1 # CORRECT : fit sur train, transform sur test
2 scaler = StandardScaler()
3 X_train_scaled = scaler.fit_transform(X_train) # fit + transform
4 X_test_scaled = scaler.transform(X_test)      # transform seul
5
6 # Sauvegarde du scaler pour la production
7 joblib.dump(scaler, 'models/preprocessor.pkl')

```

Imputation des Valeurs Manquantes

Deux stratégies d'imputation sont appliquées selon le type de variable :

- **Variables numériques** : `SimpleImputer(strategy='median')`
La médiane est préférée à la moyenne car la distribution de l'âge et des montants est souvent asymétrique (skewed). La médiane est plus robuste aux valeurs aberrantes.
- **Variables catégorielles** : `SimpleImputer(strategy='most_frequent')`
Le mode (valeur la plus fréquente) préserve la distribution naturelle des catégories.

Encodage des Variables Catégorielles

Les 12 variables catégorielles restantes sont encodées via `LabelEncoder` :

Listing 6 – Encodage catégoriel avec sauvegarde des mappings

```

1 from sklearn.preprocessing import LabelEncoder
2
3 label_encoders = {}
4 for col in categorical_columns:
5     le = LabelEncoder()
6     X[col] = le.fit_transform(X[col].astype(str))
7     label_encoders[col] = le # Sauvegarde pour inverse_transform
8
9 joblib.dump(label_encoders, 'models/label_encoders.pkl')

```

Le `LabelEncoder` est préféré au One-Hot Encoding car, avec 38 features et une cardinalité élevée (Country : 37+ valeurs), le One-Hot encoding créerait une matrice trop sparse. Le `LabelEncoder` est suffisant pour les algorithmes basés sur les arbres de décision comme le Random Forest.

Modélisation : Les Deux Techniques

Cette section constitue le cœur du projet. Deux techniques complémentaires sont implémentées, couvrant l'apprentissage non supervisé et supervisé.

Technique 1 : Clustering Non Supervisé (K-Means)

Objectif et Principe

Le clustering vise à segmenter la clientèle en groupes homogènes sans connaissance préalable de la cible. C'est une technique d'**apprentissage non supervisé** : l'algorithme découvre lui-même la structure sous-jacente des données.

Application métier : Identifier naturellement les segments Champions, Fidèles, Potentiels et Dormants pour personnaliser les campagnes marketing et allouer les ressources de rétention de manière ciblée.

Algorithme K-Means

L'algorithme K-Means minimise la variance intra-cluster selon la fonction objectif :

$$J = \sum_{k=1}^K \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \boldsymbol{\mu}_k\|^2 \quad (1)$$

où C_k est le cluster k et $\boldsymbol{\mu}_k$ son centroïde. L'algorithme alterne entre l'assignation de chaque point au centroïde le plus proche et la mise à jour des centroïdes jusqu'à convergence.

Listing 7 – Configuration K-Means pour la segmentation RFM

```

1 from sklearn.cluster import KMeans
2
3 kmeans = KMeans(
4     n_clusters=4,      # 4 segments : Champions, Fideles, Potentiels,
5     random_state=42,   # Reproductibilite
6     n_init=10,         # 10 initialisations pour eviter les minima locaux
7     max_iter=300       # Convergence garantie
8 )
9 clusters = kmeans.fit_predict(X_train)
10 joblib.dump(kmeans, 'models/kmeans_model.pkl')
```

Le choix de $k = 4$ a été validé par deux méthodes complémentaires :

- **Méthode du coude (Elbow Method)** : identification du point d'inflexion de l'inertie intra-cluster en fonction de k
- **Coefficient de silhouette** : mesure de la cohésion intra-cluster et de la séparation inter-cluster

Résultats et Interprétation

Cette segmentation permet aux équipes marketing d'adapter leurs actions : récompenser les clients actifs (Cluster 1), réactiver les clients à risque (Cluster 2), et nourrir la relation avec les clients réguliers (Cluster 0).

TABLE 6 – Répartition des clusters sur l'ensemble d'entraînement

Cluster	Effectif	Interprétation
0	1 236	Clients réguliers (fidélité moyenne, achats espacés)
1	1 276	Clients actifs (achats récents et fréquents, forte valeur)
2	980	Clients à risque (inactivité croissante, candidats au churn)
3	5	Outliers (comportements atypiques, traitement séparé)

Technique 2 : Classification Supervisée (Random Forest)

Objectif et Principe

La classification supervisée prédit la variable cible Churn (0 ou 1) à partir des features comportementales. Contrairement au clustering, l'algorithme apprend sur des exemples étiquetés pour généraliser à de nouveaux clients.

Application métier : Détecter précocement les clients sur le point de partir pour intervenir avec des offres de rétention personnalisées avant le départ définitif.

Choix de l'Algorithme : Random Forest

Le Random Forest est un ensemble d'arbres de décision entraînés indépendamment. Chaque arbre vote et la majorité l'emporte :

$$\hat{y} = \text{mode}\{h_1(\mathbf{x}), h_2(\mathbf{x}), \dots, h_B(\mathbf{x})\} \quad (2)$$

Les avantages de cet algorithme pour notre contexte sont multiples :

- Gère naturellement les features numériques et catégorielles encodées
- Robuste aux outliers grâce au bootstrap aggregating (bagging)
- Fournit l'importance des features (interprétabilité métier)
- Peu sensible aux hyperparamètres par défaut
- Pas de contrainte stricte de normalisation

Listing 8 – Random Forest avec gestion du déséquilibre de classes

```

1 from sklearn.ensemble import RandomForestClassifier
2
3 clf = RandomForestClassifier(
4     n_estimators=100,          # Nombre d'arbres dans la forêt
5     max_depth=8,              # Profondeur maximale (regularisation)
6     class_weight='balanced',  # Pondere les classes inversement
7     min_samples_leaf=10,      # Minimum d'échantillons par feuille
8     random_state=42,
9     criterion='gini'
10 )
11 clf.fit(X_train_scaled, y_train)
12 joblib.dump(clf, 'models/churn_classifier.pkl')
```

Le paramètre `class_weight='balanced'` est essentiel : il pondère chaque classe inversement à sa fréquence. La classe minoritaire (Churn=1, 33%) reçoit un poids de 2.0, forçant l'algorithme à mieux apprendre à la détecter.

Évaluation des Performances

TABLE 7 – Rapport de classification détaillé sur le jeu de test (875 clients)

Classe	Précision	Rappel	F1-Score	Support
Fidèle (0)	1.00	1.00	1.00	584
Parti (1)	1.00	1.00	1.00	291
Accuracy globale	99,9%			
Macro avg	1.00	1.00	1.00	875

Les métriques d'évaluation sont définies comme suit :

$$\text{Précision} = \frac{TP}{TP + FP}, \quad \text{Rappel} = \frac{TP}{TP + FN}, \quad F_1 = 2 \times \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}} \quad (3)$$

Analyse de l'accuracy de 99,9% : Cette performance quasi-parfaite s'explique par la nature synthétique du dataset, aux patterns très distincts et peu bruités. En production réelle, avec des données comportementales changeantes et du bruit naturel, on attendrait une accuracy de **85–95%**, ce qui resterait excellent pour un usage métier.

Importance des Features

Listing 9 – Extraction et visualisation de l'importance des features

```

1 importances = pd.DataFrame({
2     'feature': X_train.columns,
3     'importance': clf.feature_importances_
4 }).sort_values('importance', ascending=False)
5
6 # Sauvegarde de la visualisation
7 importances.head(10).plot(kind='barh')
8 plt.savefig('reports/feat_imp.png', dpi=150, bbox_inches='tight')
```

Les cinq features les plus discriminantes pour la prédiction du churn sont :

1. **Recency** : Un dernier achat récent indique une relation active
2. **Frequency** : Le nombre d'achats mesure l'engagement comportemental
3. **MonetaryTotal** : Les dépenses totales reflètent la valeur du client
4. **ReturnRatio** : Un taux de retours élevé signale le mécontentement
5. **CancelledTransactions** : Les annulations révèlent une friction croissante

Ces résultats confirment que les indicateurs RFM restent les signaux les plus puissants pour anticiper le churn, validant l'approche marketing traditionnelle par la data science. La visualisation complète est disponible dans `reports/feat_imp.png` et le rapport détaillé dans `reports/training_report.json`.

Déploiement : Interface Flask

Architecture de l'Application Web

L'application Flask sert de couche de présentation entre les modèles ML entraînés et les utilisateurs métier (équipes marketing, CRM). Elle expose deux interfaces complémentaires :

1. **Interface Web** : Formulaire HTML pour les utilisateurs non-techniques
2. **API REST** : Endpoints JSON pour intégration avec le CRM existant

Chargement des Modèles en Production

Au démarrage de l'application, les modèles et le preprocessor sont chargés une seule fois en mémoire pour éviter des rechargements coûteux à chaque requête :

Listing 10 – Chargement des artefacts ML dans Flask

```

1 import joblib
2 from pathlib import Path
3
4 BASE_DIR    = Path(__file__).parent.parent
5 MODELS_DIR  = BASE_DIR / 'models'
6
7 # Chargement unique au démarrage (pas à chaque requete)
8 preprocessor = joblib.load(MODELS_DIR / 'preprocessor.pkl')
9 kmeans       = joblib.load(MODELS_DIR / 'kmeans_model.pkl')
10 classifieur  = joblib.load(MODELS_DIR / 'churn_classifieur.pkl')
11
12 print("Les modeles sont prêts pour les predictions")

```

Mode Simplifié : 6 Features Essentielles

Pour l'utilisateur métier, demander 38 champs serait contre-productif. Un formulaire simplifié ne demande que les 6 features RFM et démographiques les plus essentielles :

Listing 11 – Features demandées à l'utilisateur vs features complètes

```

1 ESSENTIAL_FEATURES = [
2     'Recency',      # Jours depuis dernier achat
3     'Frequency',    # Nombre d'achats
4     'MonetaryTotal', # Dépenses totales
5     'MonetaryAvg',  # Panier moyen
6     'Age',          # Age du client
7     'Country',      # Pays (encode)
8 ]
9
10 # Les 32 features restantes sont completees par les moyennes historiques
11 X_train_mean = pd.read_csv('data/train_test/X_train.csv').mean()

```

Le processus de prédiction suit les étapes suivantes :

1. L'utilisateur saisit 6 valeurs dans le formulaire HTML
2. L'application crée un DataFrame avec ces 6 valeurs
3. Les 32 features manquantes sont complétées par les moyennes historiques
4. Le **StandardScaler** (fit sur le train) transforme les 38 features
5. Les 2 modèles prédisent : Cluster RFM et probabilité de Churn
6. Les résultats sont affichés avec code couleur (vert / orange / rouge)

Exemple de Résultat d'Analyse

Pour un client présentant les caractéristiques suivantes :

- Recency = 400 jours (absent depuis longtemps)
- Frequency = 1 (un seul achat historique)
- MonetaryTotal = 30£ (faible dépense)

TABLE 8 – Résultat d'analyse pour un client à haut risque

Technique ML	Résultat	Action recommandée
Clustering (K-Means)	Groupe 2 (Dormants)	Campagne de réactivation
Classification (RF)	PARTI (100% risque)	Offre de rétention urgente

Compétences Acquises et Mapping Pédagogique

Ce projet couvre exhaustivement les compétences du cahier des charges :

TABLE 9 – Mapping des compétences pédagogiques avec les réalisations concrètes

Compétence	Fichier source	Réalisation détaillée
Exploration	exploration.py	Analyse des 52 features, détection des valeurs manquantes (Age : 30%), identification des formats inconsistants (dates dans Monetary), visualisation des distributions, matrice de corrélation, analyse de la distribution du Churn
Préparation	src/preprocessing.py	Nettoyage des types corrompus, suppression de 13 features causant du data leakage, imputation médiane/mode, encodage LabelEncoder, split stratifié 80/20, normalisation StandardScaler sans fuite de données
Transformation	src/preprocessing.py	Réduction de dimension (52 → 38 features), normalisation centrée-réduite, création des variables agrégées pour le clustering RFM
Modélisation	src/train_model.py	Implémentation des 2 techniques : K-Means (4 clusters RFM), Random Forest Classifier (prédiction Churn). Sauvegarde des artefacts : <code>kmeans_model.pkl</code> , <code>churn_classifier.pkl</code> , <code>preprocessor.pkl</code>
Évaluation	src/train_model.py	Classification : accuracy, précision, rappel, F1, matrice de confusion (<code>reports/confusion_matrix.png</code>). Importance des features (<code>reports/feat_imp.png</code>). Rapport complet dans <code>reports/training_report.json</code> . Interprétation critique des résultats (99,9% lié à la nature synthétique)
Déploiement	app/app.py	Interface web Flask avec formulaire simplifié (6 features), chargement des modèles .pkl, prédictions temps réel, affichage des 2 résultats (cluster RFM, churn) avec code couleur

Conclusion et Perspectives

Bilan du Projet

Ce projet nous a permis de maîtriser la chaîne complète du Machine Learning appliqué au retail, depuis l'exploration de données brutes et imparfaites jusqu'au déploiement opérationnel via une API web. Les deux techniques fondamentales (clustering et classification) ont été implémentées et évaluées avec rigueur.

Les points forts de notre approche :

- **Prévention du data leakage** : Identification et suppression de 13 features problématiques avant toute modélisation
- **Gestion du déséquilibre** : Utilisation de `class_weight='balanced'` pour la classification
- **Reproductibilité** : Environnement virtuel, `random_state` fixé, pipeline structuré et versionné
- **Interprétabilité** : Analyse de l'importance des features et segmentation métier actionable
- **Traçabilité** : Rapport d'entraînement sauvegardé dans `reports/training_report.json`

Limites et Améliorations Futures

Plusieurs axes d'amélioration ont été identifiés pour une mise en production réelle :

1. **SMOTE** : Technique de sur-échantillonnage synthétique pour renforcer la classe minoritaire si le déséquilibre s'aggrave
2. **GridSearchCV / Optuna** : Optimisation automatique des hyperparamètres (`max_depth`, `n_estimators`, `min_samples_leaf`)
3. **SHAP / LIME** : Explicabilité des prédictions au niveau individuel (conformité RGPD, droit à l'explication)
4. **Monitoring du drift** : Détection automatique de la dégradation des performances dans le temps (concept drift)
5. **Feature Store** : Centralisation et versionnement des features pour tous les modèles
6. **Tests A/B** : Validation de l'impact business des campagnes de rétention ciblées

Apports Pédagogiques

Ce projet a consolidé notre compréhension des enjeux réels du Machine Learning en entreprise :

- Les données réelles sont toujours imparfaites et nécessitent un travail de nettoyage substantiel
- Le data leakage est une erreur subtile mais critique qui invalide un modèle
- La compréhension métier est aussi importante que la technique (savoir interpréter RFM et churn)
- Le déploiement est aussi difficile que la modélisation (passage du notebook à la production)

Références

- [1] F. Pedregosa et al., *Scikit-learn : Machine Learning in Python*, Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

- [2] A.M. Hughes, *Strategic Database Marketing*, McGraw-Hill, 1994. (Chapitre sur la segmentation RFM)
- [3] A. Ahmad et al., *Customer Churn Prediction in Telecommunication Industry Using Machine Learning*, IEEE Access, vol. 7, 2019.
- [4] Pallets Projects, *Flask Documentation*, <https://flask.palletsprojects.com/>, 2024.
- [5] L. Breiman, *Random Forests*, Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
- [6] T. Hastie, R. Tibshirani, J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009.